

Go Fundamentals for Distributed Algorithms III

Danilo Kaćanski

Faculty of Technical Sciences, Novi Sad
Applied Algorithms in Control Systems

March 2026.

Functions, methods, closures and pointers

Go Functions and Methods

- **Functions** define general-purpose behavior
 - They are **not tied** to any specific data type
 - We use them for standalone logic (e.g. math, utilities, helpers)
- **Methods** attach behavior to a specific type usually struct
 - They represent actions that **belong to that data**
 - Also they improve code organization and readability
- **When to use what?**
 - Use a **function** when logic is independent of a type
 - Use a **method** when behavior conceptually belongs to a type
- **Example intuition:**
 - `add(a, b) → function` (generic operation)
 - `account.Deposit() → method` (behavior of Account)

Basic Function Structure

Function declaration and call

```
// func -> keyword to define a function  
// add -> function name  
// (a int, b int) -> parameters with types  
// int -> return type  
func add(a int, b int) int {  
    return a + b  
}  
func main() {  
    result := add(2, 3)  
    fmt.Println(result)  
}
```

- Function is a combination of: name + parameters + return type + body
- Arguments are the actual values passed during the call
- Return value is sent back to the caller

Multiple Returns

Result + error is common pattern in GO

```
func divide(a, b float64) (float64, error) {  
    if b == 0 {  
        return 0, fmt.Errorf("division by zero")  
    }  
    return a / b, nil  
}
```

- Functions can return **multiple values**
- Common pattern: (result, error)
- nil means **no error occurred** and caller is responsible for checking that

Why is this important?

- Go does **not rely on exceptions** for normal errors
- Errors are handled **explicitly and locally**
- Makes control flow **predictable and readable**

Variadic Functions

A function can accept zero or more values

```
func sum(nums ...int) int {  
    total := 0  
    for _, n := range nums {  
        total += n  
    }  
    return total  
}
```

- `...int` means the function can receive **any number of int values**
- This means that all of these work: `sum(1, 2)`, `sum(1, 2, 3, 4)`, and even `sum()`

Important

- A variadic parameter must be the **last parameter** in the function signature.

Functions as Values

Passing behavior as a parameter

```
// op is a function parameter: func(int, int) int  
func apply(a, b int, op func(int, int) int) int {  
    return op(a, b) // calling the passed function  
}  
  
// regular function matching the required signature  
func mul(x, y int) int {  
    return x * y  
}
```

- Functions in Go are **first-class values**
- They can be:
 - passed as arguments
 - stored in variables
 - returned from functions

Custom Function Types I

Named function type for reuse

```
type Operation func(int, int) int // Operation is a type alias for a function
func compute(a, b int, op Operation) int { // op must match the Operation sig
    return op(a, b)
}
func sub(x, y int) int { // this function matches Operation
    return x - y
}
```

- Operation defines a **function type**
- It represents any function that:
 - takes two int values
 - returns one int
- sub is compatible because it has the same signature

Why use custom function types?

- It improves **readability** (no long function signatures everywhere)
- It can make intent clearer: `Operation` describes purpose, not just shape
- Easier to reuse across multiple functions

Best use cases:

- When multiple functions share the same signature
- When passing behavior around (e.g. calculators, strategies)
- When building higher-level abstractions

Returning Functions I

Choosing behavior based on input

```
type Operation func(int, int) int

func choose(op string) Operation {
    if op == "+" {
        return func(a, b int) int { return a + b }
    }
    return func(a, b int) int { return a - b }
}
```

- Functions in Go can **return other functions**
- The returned value represents **behavior**, not just data so op in this case:
op := choose("+") is a function → op(3,4)

Closure definition

```
func makeCounter() func() int {  
    count := 0 // local variable  
  
    return func() int { // closure  
        count++ // uses outer variable  
        return count  
    }  
}
```

- The inner function **captures** the variable count
- count is not destroyed when the outer function returns as it may be expected
- Each closure has its own **persistent state**

Important

A closure is a function that **remembers variables from its outer scope**.

Using the counter

```
counter := makeCounter()

fmt.Println(counter()) // 1
fmt.Println(counter()) // 2
fmt.Println(counter()) // 3
```

- `makeCounter()` returns a **function**
- That function remembers the variable `count`
- Each call updates the same count

Key Idea

The returned function keeps its own internal state.

Execution order example

```
func demo() {  
    defer fmt.Println("first") // pushed first  
    defer fmt.Println("second") // pushed second  
    fmt.Println("body") // runs immediately  
}
```

What gets printed?

- Step 1: print "body"
- Step 2: execute deferred calls (in reverse order)
- Output: body → second → first

- **Defer** stacks calls (like a stack)
- Executed in **LIFO order** (Last-In, First-Out)
- Run **when function is about to return**

Important

`os.Exit(...)` terminates the program immediately → deferred calls are NOT executed.

Deferred Closures I

Execution trace

```
func demo() {  
    x := 10  
    defer fmt.Println(x) // (1) argument evaluated NOW 10  
    defer func() {  
        fmt.Println(x) // (2) uses x later  
    }()  
    x = 20 // value changes  
} // function returns deferred calls run
```

Output

20 → 10

- Deferred calls execute in **reverse order**
- Closure runs first → sees `x = 20`
- Then `fmt.Println(x)` → uses saved value 10

- **Regular defer call:**
 - Arguments are evaluated **immediately**
 - Value is stored at defer time
- **Deferred closure:**
 - Captures the **variable itself**
 - Value is read **later (at execution time)**

Mental Model

- `fmt.Println(x)` → stores **value (10)**
- `func() { fmt.Println(x) }` → stores **reference to x**

Pointers Basics

Address and dereference operators

```
func main() {  
    x := 5 // normal integer variable  
    p := &x // p stores the address of x  
    fmt.Println(*p) // read value at that address prints 5  
    *p = 8 // write through pointer: change value stored at x's address  
    fmt.Println(x) // x is now 8  
}
```

- `&x` means: **take the address of x**
- `p` does not store the value 5, it stores **where 5 is located**
- `*p` means: **go to that address and access the value**
- Nil pointer dereference causes a runtime panic

Key Idea

A pointer lets us read or modify the original variable through its memory address.

Pointer-Based Updates

Modify caller data through helper functions

```
type Account struct {  
    Balance int  
}  
  
func deposit(a *Account, amount int) {  
    // a is a pointer it holds address of original struct  
    a.Balance += amount  
    // same as: (*a).Balance += amount  
    // go to the original Account and update its Balance  
}
```

- a *Account means we pass a **pointer**, not a copy
- The function works on the **original object in memory**
- a.Balance is shorthand for (*a).Balance
- Changes inside the function are **visible outside**

Slices and Maps in Functions

Shared mutation and append caveat

```
func touch(s []int, m map[string]int) {  
    s[0] = 99 // modifies underlying array, so the caller sees change  
    m["x"] = 1 // modifies shared map, so the caller sees change  
    s = append(s, 7) // may create NEW array, so only local slice is updated  
}
```

- Slice **s** is a **descriptor** (pointer + len + cap) so `s[0] = 99` changes the **shared underlying array**
- Maps work same way
- `append` may create a **new array** and reassign **s**

Defining a method

```
type Account struct {  
    Balance int  
}  
  
func (a Account) getBalance() int {  
    return a.Balance  
}
```

- A **method** is a function with a **receiver** (in this case (a Account)) which is the object that the method belongs to
- It is called using dot notation

Usage

```
acc := Account{Balance: 100}  
fmt.Println(acc.getBalance())
```

Value vs Pointer Receiver

Two ways to define methods

```
// value receiver (copy)
func (a Account) deposit_c(amount int) {
    a.Balance += amount // modifies COPY
}

// pointer receiver (original)
func (a *Account) deposit_o(amount int) {
    a.Balance += amount // modifies ORIGINAL
}
```

- Value receiver → method works on a **copy**
- Pointer receiver → method works on the **original object**

Important

Only pointer receivers can modify the original struct.

Exercises

Task 1: Closures as State

Task 1

Implement a function:

- `makeBankAccount(initial int)`
- It should return a function: `func(int) int`

The returned function should:

- Add the input amount to the balance
- Return the updated balance

Test scenario:

- Create two accounts
- Perform multiple updates on both
- Print balances after each operation

Task 2: Copy vs Original

Task 2

Given:

Struct

```
type Account struct {  
    Balance int  
}
```

- Implement two methods:
 - `depositValue(amount int)`
 - `depositPointer(amount int)`
- Call both methods on the same object
- Print balance after each call

Task 3: Defer and Closures

Task 3

Write a function:

Starter

```
func test() {  
    x := 10  
    // TODO 1: add a defer that prints x directly  
    // TODO 2: add a defer with a closure that prints x  
    // TODO 3: change x to 20 before function ends  
}
```

- Call `test()` from `main`
- Run the program and observe the output